



成绩 _____

北京航空航天大学
BEIHANG UNIVERSITY

微机原理及接口技术 实验报告

院（系）名称	自动化科学与电气工程学院
专业名称	自动化
学生学号	22375413
学生姓名	夏思博
指导教师	林新

2024 年 12 月

实验三 数/模与模/数转换

实验时间 2024,12.12 实验编号 同组同学 王梓枫

一、实验目的

1. 通过实验了解数模转换的原理，使用典型的数/模转换芯片 DAC0832，在单缓冲工作方式下，通过编写汇编程序来控制数/模转换，理解不同输出电路连接方式下电压输出值(单极性、双极性)与输出数字量之间的对应关系。;掌握 ILE, CS 以及 WR1 信号对输入寄存器的控制作用,掌握 WR2 和 XFER 信号对 DAC 寄存器的控制作用;

2. 建立模数转换的感性认识,加深对连续模拟量转换为离散数字量的理解;掌握 ADC0809 的转换使用方法,包括模数转换的启动、等待转换结束、读取转换结果整个过程;编写汇编程序对 AD 转换后的数字量进行处理并显示,提高微机原理与接口技术课程的综合应用能力。

3. 通过实验加深理解过程控制系统中计算机与外界进行数据交互的过程。

二、实验原理

1. 说明实验中涉及到的相关理论、概念、算法或者原理

(一) 数模转换

实验电路连线:

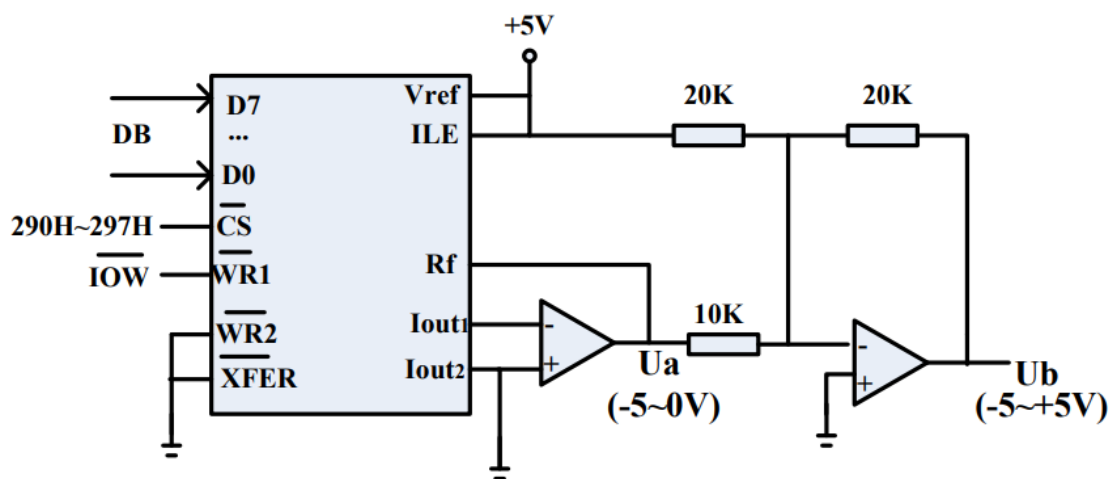


图 1. 数模转换实验电路连接图

图 1 为数/模转换实验电路连线图, DAC0832 采用单缓冲方式, 具有单双极性输

出端(图中的 U_a 为单极性、 U_b 为双极性)。

片选段 CS 连接地址译码器对应 290H-297H 的输出端,当地址信号为 290H-297H 时, DAC0832 的 CS 端被选通,为低电平有效。

1) 产生方波信号的数字量

要产生方波输出信号,在数据段中存放方波的数字量,要使方波输出在 8 位 DA 的数字量范围 ($0 \sim 255$) 内。取两个点分别作为方波的高电平和低电平输出。若在 U_a 端产生方波,则高电平取数字量 0(0V),低电平取数字量 255(-5V);若在 U_b 端产生方波,则高电平取数字量 255(+5V),低电平取数字量 0(-5V)。

2) 产生正弦信号的数字量

要产生正弦波的输出信号,需要从一个周期中取多个点输出。下面以 20 个点为

例来说明。假设 $Y = \sin n (2\pi/20)$, $n=0-19$,为了使输出在 8 位 DA 的数字量范围 ($0 \sim 255$) 内, 32π 对 Y 进行放大和上移。设 $Y = 80H + 80H \times \sin n (2\pi/20) = 80H(1 + \sin n (2\pi/20))$, 列出 $n=0-19$ 时 Y 的输出情况(超过 FFH,则取值 255)。

n	0	1	2	3	4	5	6	7	8	9
Y	128	168	203	232	250	255	250	232	203	168
n	10	11	12	13	14	15	16	17	18	19
Y	128	88	53	24	6	0	6	24	53	88

图 2. 正弦量的数字量取值

把要输出的数据放在数据段中,循环输出数据,将要输出的数据存在数据段中,循环输出数据以便显示方波或正弦波。程序中可以通过按键来切换波形或者退出循环结束程序。

(二) 模数转换

1) 实验硬件连接

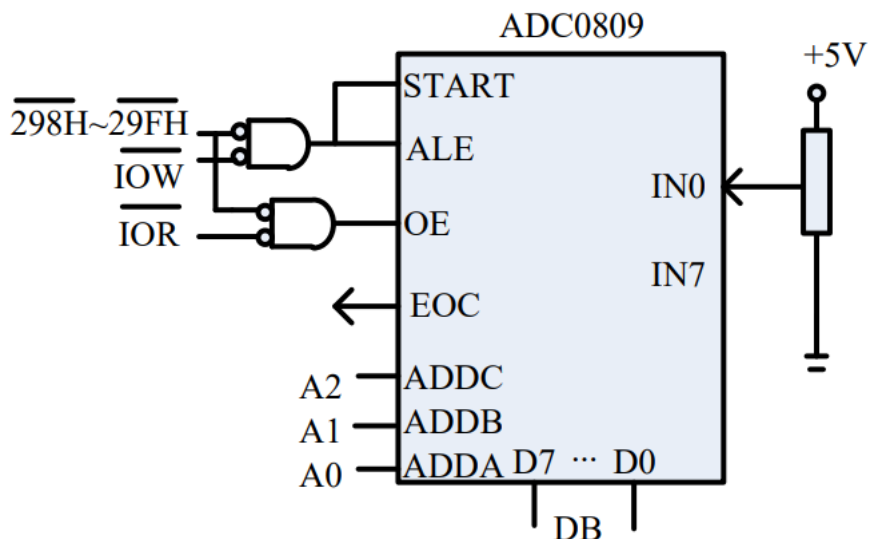


图 3. 模数转换硬件连接图

①ADC0809 的模拟量输入端 IN0 与 电位计 RW1 的电压输出端 U1 (位于实验箱左下方, 小键盘旁边)

②ADC0809 的片选信号 CS 与计算机端口控制端 298H~29FH

2) 编程

①ADC0809 转换过程

启动转换和输入转换后的数据, 都是将端口号写入 DX 后直接使用 OUT 指令和 IN 指令即可。

等待转换结束有两种方法。一种是查询 ADC0809 的 EOC 状态, 另一种是长时间延时。由于实验硬件连线不支持 EOC 状态查询, 因此本实验采用后一种方法, 通过编写和调用延时子程序 DELAY 等待 ADC0809 转换结束。

②显示电压值:

电压值显示算法的原理如下: 采集到的数据 X 为 8 位自然二进制编码, 范围 0~255.

对应电压为 0~+5V

因此, 根据除法计算规则, 需要显示的个位数为: $I = X * 5 / 255$; 此外还需要保持余数 $J = X * 5 \% 255$, 以便计算小数点后面的电压数据。

小数点后面第一位数据: $K = J * 10 / 255$; 另有余数 $M = J * 10 \% 255$ 小数点后面第二位

数据: $N=M*10/255$

③循环方法:

由于电压值需要反复读取和输出,因此程序需要循环操作。为了避免循环过快,导致数据显示快速刷新无法看清,因此需要对循环的速度和节奏加以控制。最简单的方法是使用 INT21H 的 8 号功能调用。

每当一轮数据采集和显示结束后,调用 INT21H 的 8 号功能,程序暂停等待键盘的输入。如果输入的按键是 q 键则退出程序。否则重新开始一轮模数转换操作。

④延时程序

延时程序主要用于启动 ADC0809 转换后,等待转换结束。延时的原理是,每条指令的执行都需要时间,当反复执行某些指令时即可占用相当多的 CPU 之间,从而达到延时的目的。

为了加长延时时间,可以使用嵌套循环结构。

⑤显示 16 进制

转换后的数据为 8 位的数字量,需要针对高 4 位和低 4 位分别进行显示,另外也需要针对 0~9 和 A~F 做出不同的处理。

首先显示高 4 位。将模数转换后的数据逻辑右移 4 位,只保留高 4 位。然后判断此高 4 位是否大于 9。如果大于 9,则表明其数值在 A~F 之间,需要加上 37H,转换为 ASCII 码值进行输出;否则加上 30H,转换为 ASCII 码值进行输出。

然后显示低 4 位。将模数转换后的数据和 0FH 进行与操作,只保留低 4 位。按上述方法显示低 4 位。

2. 附上电路原理图、流程图



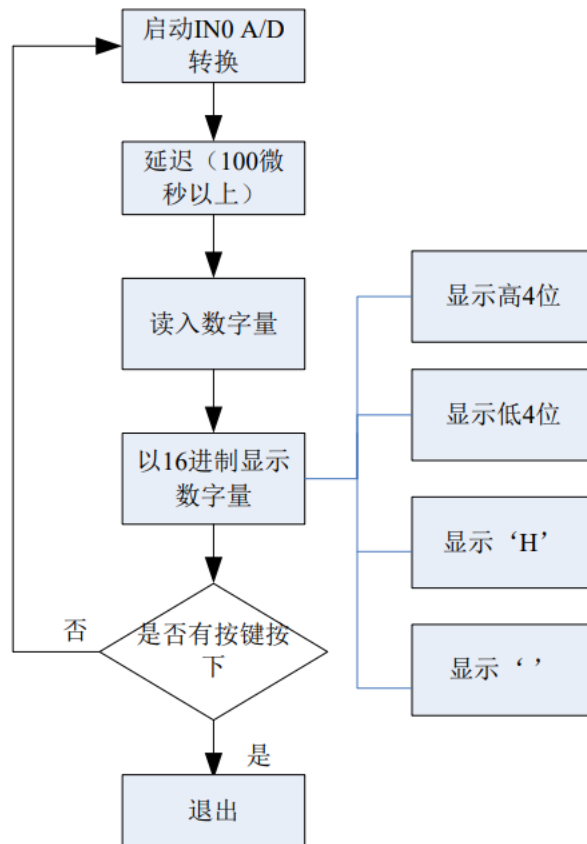


图 6. 模数转换流程图

数模转换流程：实验箱 ADC0832 数模转换按 a、b、c、d 依次输出正弦波、方波、锯齿波、三角波。

三、实验内容与实验过程描述

1、实验内容一 实验箱数模转换：

方波和正弦波是应用中常见的两种信号源，实验利用数模转换芯片 DAC0832 产生方波及正弦波。

DAC0832 采用单缓冲方式，输出端具有单双极性两种方式。

①在数据段中存放好方波（0，255）和正弦波的数字量（正弦波要求 20 个值），可以执行程序将数据段中的数字量送到 DAC0832 的输出端产生方波和正弦波。

DAC0832 对应的端口地址为 290H~297H。

②按 a、b、c、d 依次输出正弦波、方波、锯齿波、三角波。按 q 退出

2、实验内容二 实验箱模数转换：

编写程序对 ADC0809 的 IN0 通道的模拟量进行模数转换。包括:ADC 输出端口号定义，逻辑段定义，程序初始化，启动 ADC0809，延时子程序，转换数据读取;将模数转换得到的数字以合适的方式显示在微机屏幕上。显示的数字可以是以下一种或多种方式:16 进制数、二进制、十进制数或电压值。

程序应当能够连续显示转换后的数字量,可以在以下几种方法中选择其中一种:

- 1) 间隔显示。每隔一段时间即触发一次 AD 转换,读取转换后的数据并进行显示。
- 2) 使用键盘按键来控制下一次的显示。例如:每按一次空格键,就启动一次 AD 转换并将转换后的数据显示在屏幕上,当按下 ESC 键后程序退出。
- 3) 每隔一段时间启动 AD 转换,读取转换后的数据。如果读取的数据与上次相同则不显示,如果不同则显示。

3、实验内容三: Proteus 仿真 DAC0832 输出波形

- 1) 在 Proteus 中使用 DAC0832 设计搭建 DA 电路,编写程序输出三角波、锯齿波、方波和正弦波, DAC0832 端口地址 200~207H;
- 2) 通过 8255A 的 PA0-3 端口单刀四掷开关控制输出哪个波形。
- 3) 示波器连接输出端,改变单刀四掷开关,观察输出波形

4、实验内容四: Proteus 仿真 ADC0808 输入由滑动变阻器控制的 0-5V 电压值,并将电压值显示在数码管上

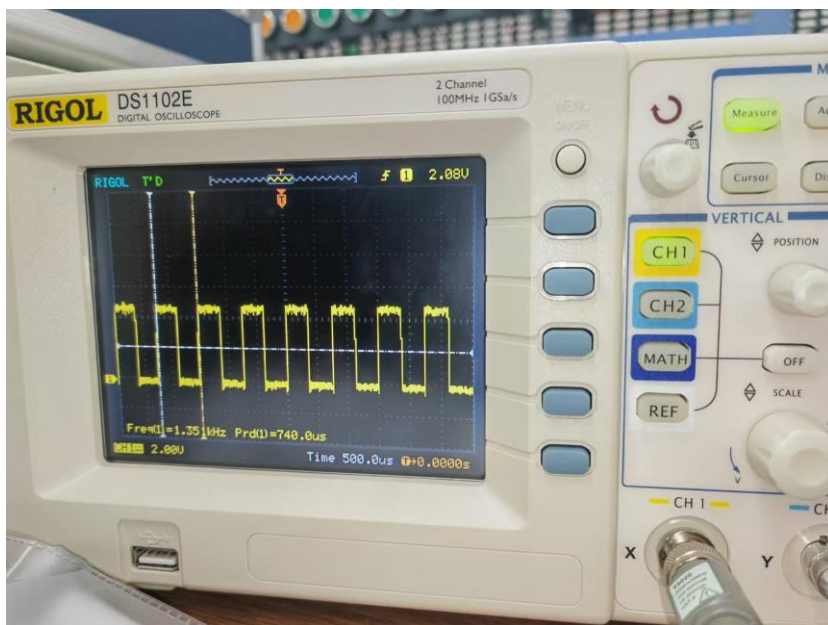
- 1) 在 Proteus 中使用 ADC0808 设计搭建 AD 电路
- 2) 数据段存储晶体管 0-9 对应七段码
- 3) 编写程序输入 ADC0808 的电压值并保存在数据段中, DAC0808 端口地址 200~207H;8255A 端口地址 238~23EH
- 4) 通过 8255A 的 PA 端口和 PC 端口将电压值转化为七段码显示在数码管上。

四、实验结果分析与实验结论

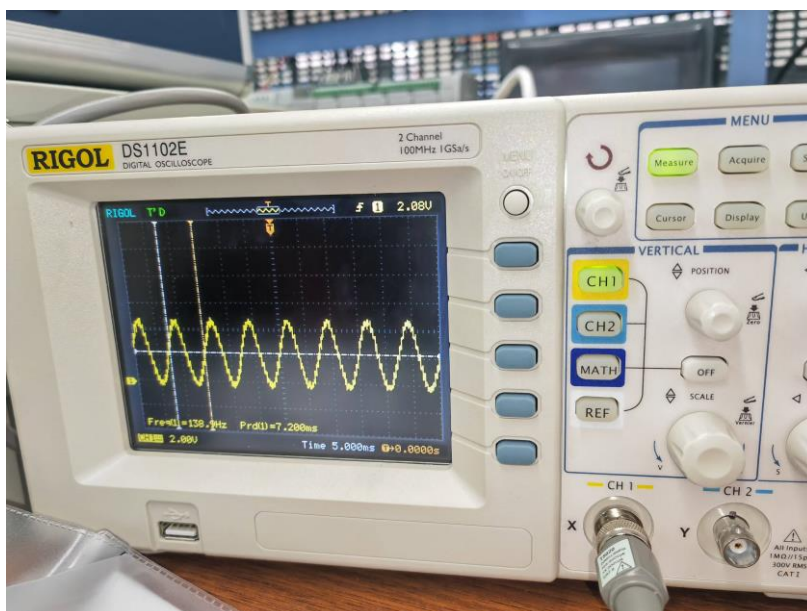
1. 对实验所得结果进行分析处理，总结实验结论。

1) 实验一 DA 转换:

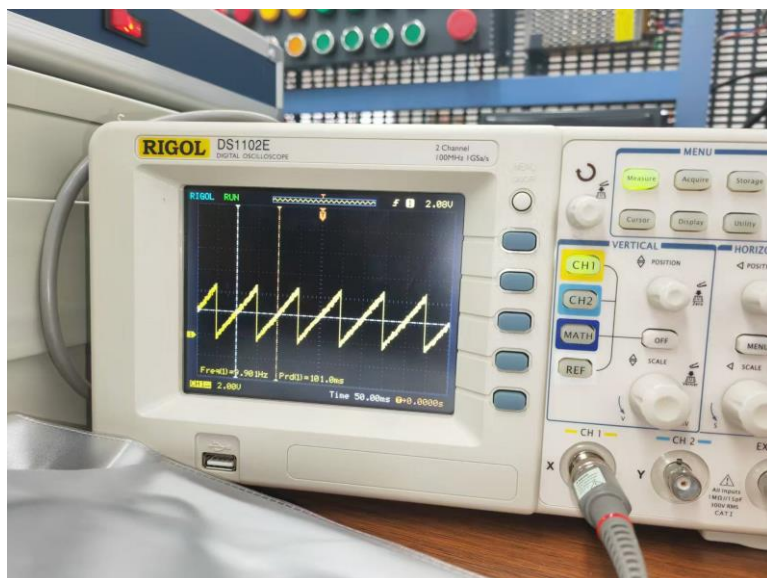
输出方波:



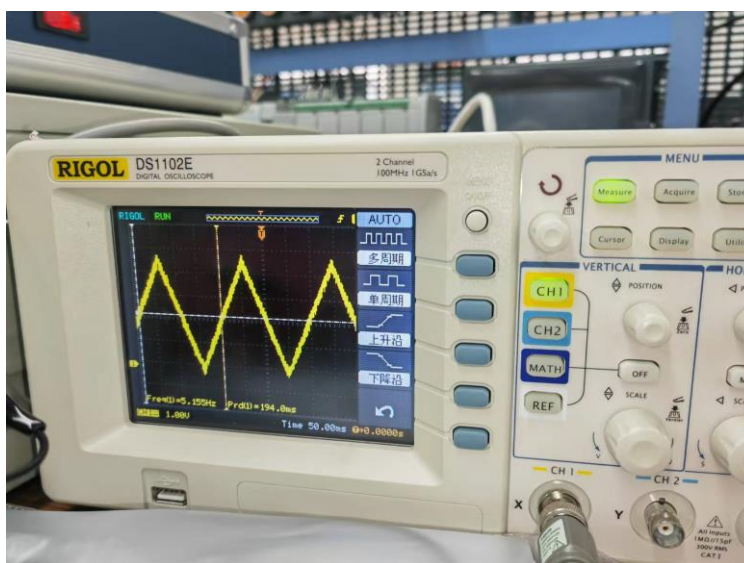
输出正弦波:



输出锯齿波:



输出三角波:

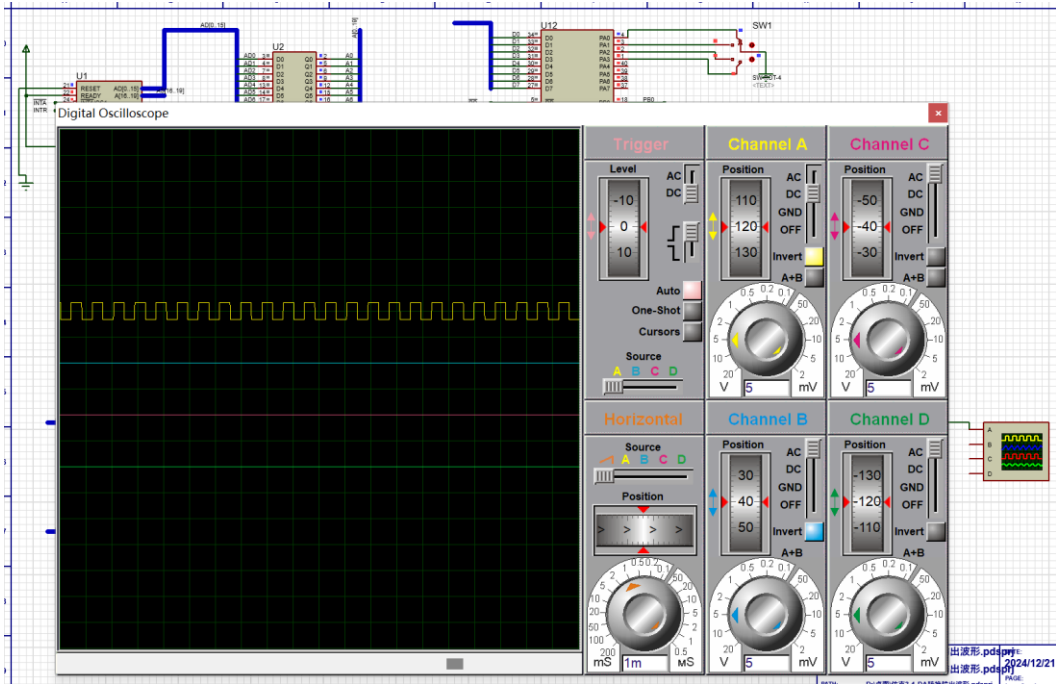


2) 实验二 AD 转换:

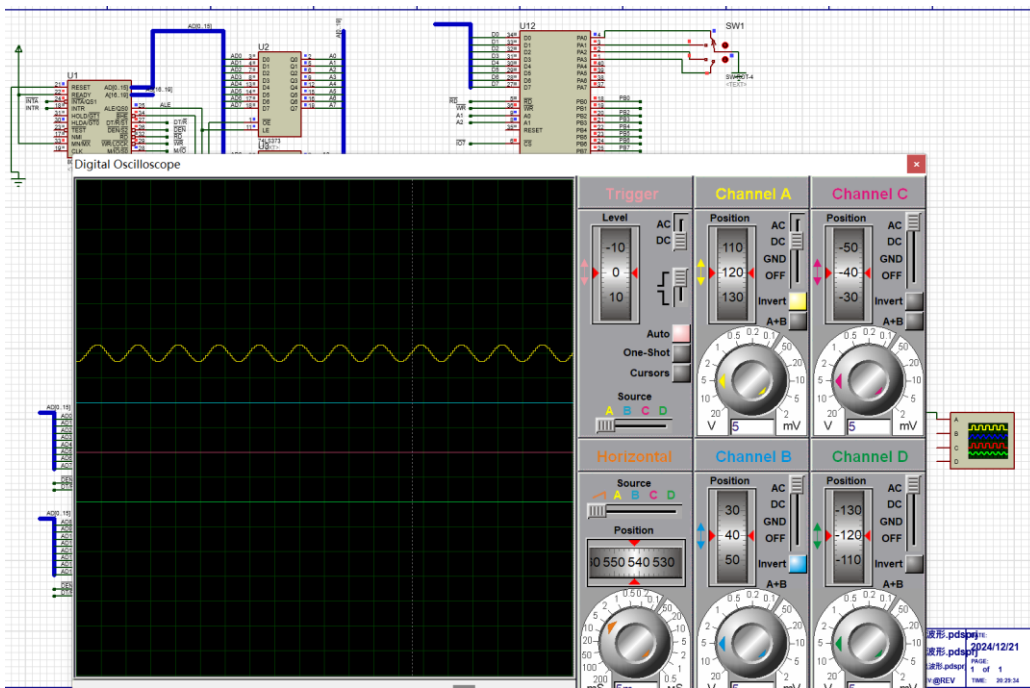
实验结果得到在屏幕中输出电压值，且精确到小数点后两位。每次输出完后按下 q 退出程序，按下其他键重新显示电压值。

3) 实验三 Proteus 仿真 DAC0832 输出波形

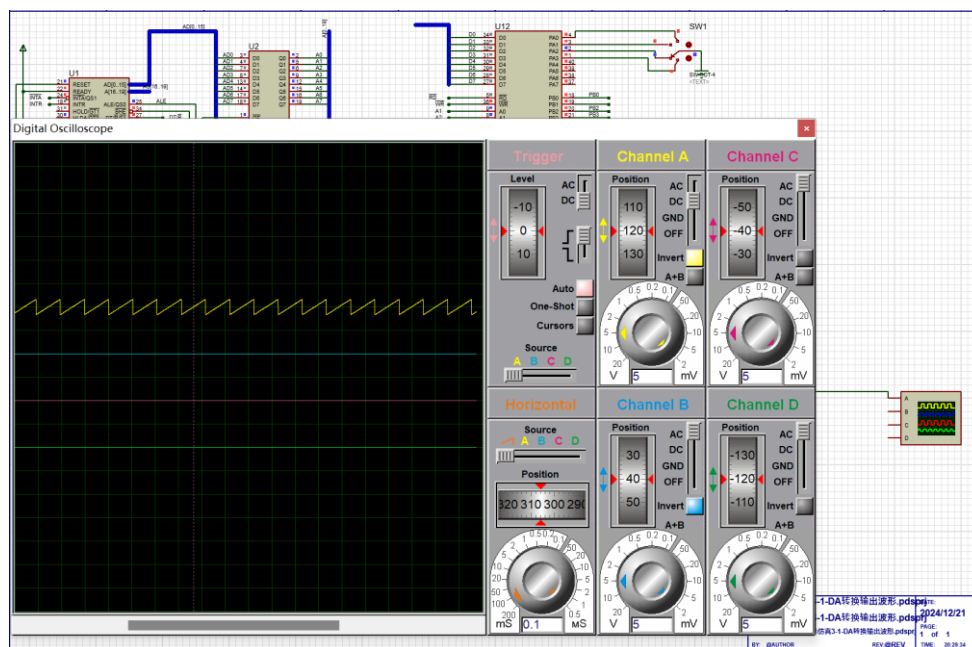
输出方波:



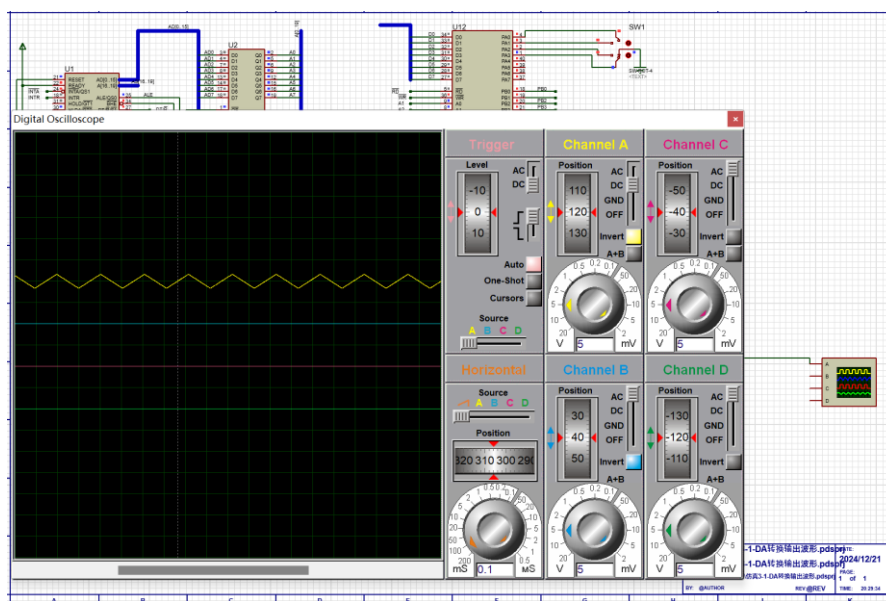
输出正弦波：



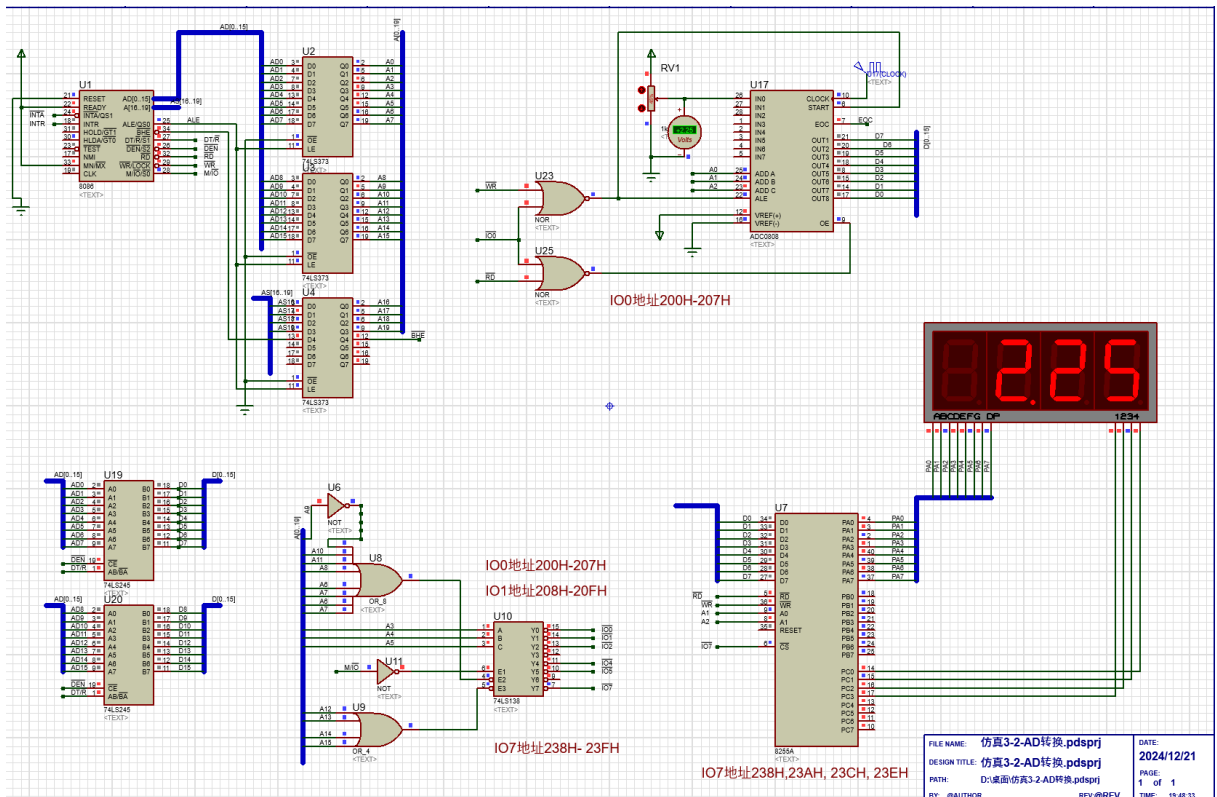
输出锯齿波：



输出三角波：



4) 实验四 ADC0808 模数转换，显示在数码管上



1. 实验过程中遇到的问题分析总结。

控制输出波形时，代码比较长，超过了 127 行，无法正常使用 JE 等条件转移，最后只能一次性控制三种波形的输出。因此，需要使用 JMP 功能，先用 JE 进行小跳转，再接 JMP 进行大跳转。

五、收获、体会及建议

1. 总结实验收获与个人体会；欢迎同学提出改善实验课程的建议。

通过本次实验我了解了 ADC0809 和 DAC0832 的工作方式，学会了如何通过 DAC0832 输出各种波形以及通过 ADC0809 实时读取电压值。在 Proteus 仿真中，我觉得 ADC0809 和 DAC0832 这两个数模/模数转换器与 8255A 接口芯片要融会贯通，联合使用，这样能达到更多的功能。

附录 1

1、预习思考题及实验验证

1) 单极性端输出电压 $V_{o1} = -V_{ref} \times D/256$

双极性端输出电压 $V_{o2} = (D - 128) \times V_{ref}/128$

其中 D 为数字量对应十进制 $D0 + D1 \times 2^1 + D2 \times 2^2 + D3 \times 2^3 + D4 \times 2^4 + D5 \times 2^5 + D6 \times 2^6 + D7 \times 2^7$

2) 有 DOS2、6、9 号功能

6 号功能当 DL=0FFH 时为从键盘输入字符，且不等待。

2 号功能显示单个字符

9 号功能显示以\$符为结尾的字符串

3)

①显示 2 进制：对字节数据进行循环左移，每移一次与 1 后加 30H 输出。循环 8 次

②显示 16 进制：先对字节进行逻辑右移 4 位，只保留高四位，之后进行判断是否大于 9，若不大于，则加 30H 后进行输出，若大于。则加 37H 后进行输出。之后在对字节数据与 0FH 保留低四位，在与 9 进行比较，若不大于，则加 30H 后进行输出，若大于。则加 37H 后进行输出。

;16进制输出

```

    PUSH AX

    SHR AL,4;高四位
    AND AL,0FH
    CMP AL,9
    JBE NUM
    ADD AL,37H;数字A-F
    JMP NEXT11
NUM:  ADD AL,30H;数字0-9

NEXT11:  MOV DL,AL
         MOV AH,02H

```

```

INT 21H
POP AX
PUSH AX

AND AL, 0FH; 低四位
CMP AL, 9
JBE NUM1
ADD AL, 37H; 数字A-F
JMP NEXT10
NUM1:  ADD AL, 30H; 数字0-9

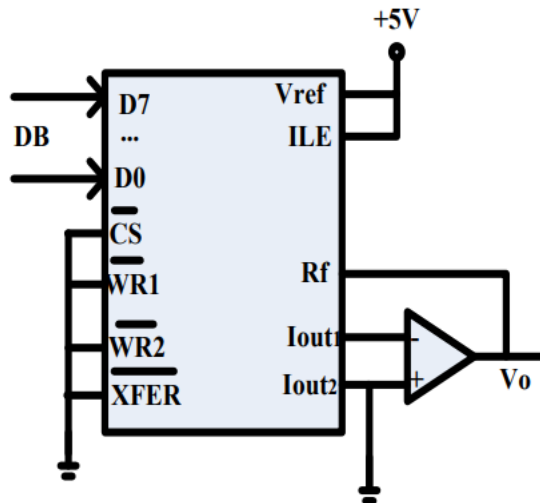
NEXT10: MOV DL, AL
        MOV AH, 02H
        INT 21H
  
```

4) 将待显示的字节数据(0~255)使用 DIV 指令除以 100, 商保存在 AL 中, 即为十进制的百位数; 余数保存在 AH 中, 余数复制到 AL 寄存器, AH 清零, 再次使用 DIV 指令除以 10, 得到的商即为十位数, 余数为个位数。然后将得到的百位、十位、个位数分别加上 30H, 转换为 ASCII 码值进行显示输出。

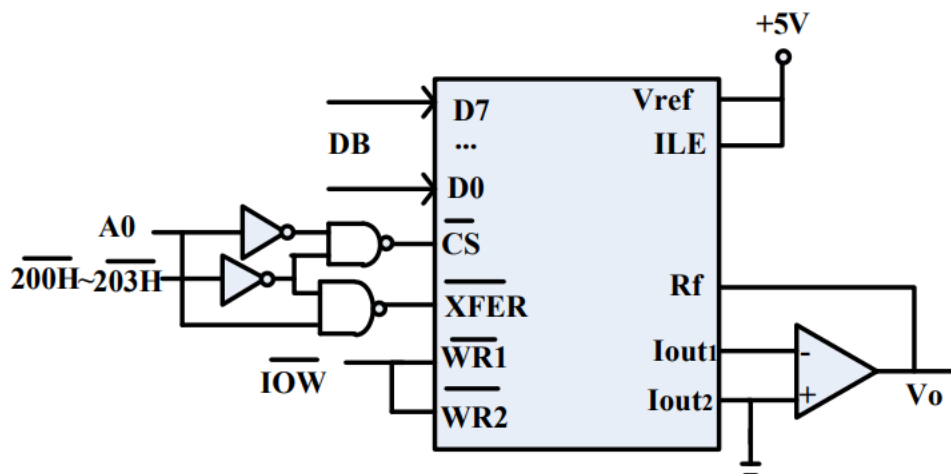
2、实验后思考题

1. 在程序中有 MOV DX, PORT0832, 修改程序中所有 PORT0832 的值, 修改成新的端口地址

2. ①直通方式连线:

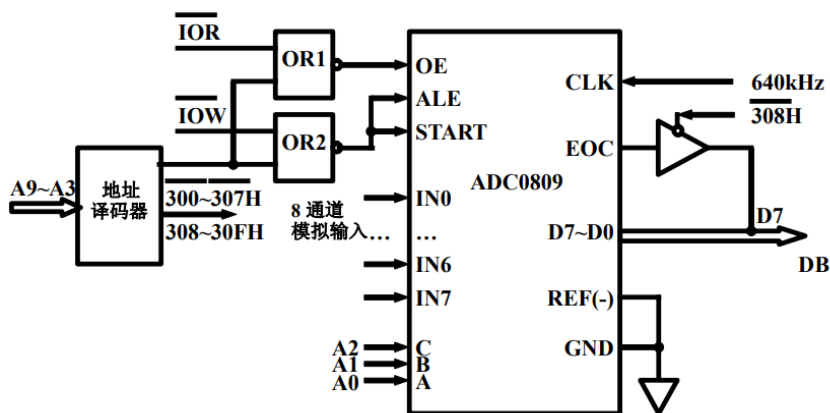


②双缓冲方式连线：



3. Ub 端输出-5 到 1V 之间的三角波

4. 硬件连接图：



相应的程序：

```
MOV DX, 308H           ;DX 指向状态口 308H
POLL: IN AL, DX         ;读入 EOC 状态
TEST AL, 80H           ;EOC (D7)=0?开始转换了?
JNZ POLL               ;非 0, 循环等待
NO_END:
IN AL, DX               ;EOC=0, 已经开始转换
TEST AL, 80H           ;再查 EOC 是否为 1
JZ NO_END               ;EOC=0, 等待转换结束
```


附录 2：实验源程序

实验一 实验箱 DAC0832 输出波形

```

DATA SEGMENT
    TBL DB
    128, 168, 203, 232, 250, 255, 250, 232, 203, 168, 128, 8
    8, 53, 24, 6, 0, 6, 24, 53, 88
    COUNT1 EQU $-TBL
    TBL2 DB 0, 255 ; 方波
    COUNT2 EQU $-TBL2

DATA ENDS

STK SEGMENT
    SSDAT DW 100 DUP(?)
STK ENDS

CODE SEGMENT
    ASSUME CS:CODE, DS:DATA, SS:STK
START:
    ; 初始化及数据段、堆栈段观察记录
    MOV AX, STK
    MOV SS, AX
    MOV SP, 2*100

    MOV AX, DATA
    MOV DS, AX
;USER CODE START

    MOV BX, 0
    CALL KEYIN
    CMP AL, 31H
    JE AGN1
    CMP AL, 32H
    JE AGN2
    CMP AL, 33H
    JE AGN3
    CMP AL, 34H
    JE AGN4

AGN1: ; 输出一个幅值为5V的正弦波
    MOV BL, 0
AGN11:
    MOV DX, 290H
    MOV AL, TBL[BX]
    OUT DX, AL
    CALL DELAY

    CALL TRAN
    INC BL
    CMP BL, COUNT1
    JNE AGN11
  
```

```

    JMP AGN1

AGN2: ; 输出一个方波
    MOV BX, 0
AGN22:
    MOV DX, 290H
    MOV AL, TBL2[BX]
    OUT DX, AL
    CALL DELAY

    CALL TRAN
    INC BL
    CMP BL, COUNT2
    JNE AGN22
    JMP AGN2
; 也可以写成如下形式:
; MOV DX, 290H
; MOV AL, 0
; OUT DX, AL
; CALL DELAY
; MOV AL, 255
; OUT DX, AL
; CALL TRAN

AGN3: ; 输出一个峰值为5V的锯齿波形
    MOV BX, 0
AGN33:
    MOV DX, 290H
    MOV AL, BL
    OUT DX, AL
    CALL DELAY

    CALL TRAN
    INC BL
    CMP AL, 255
    JB AGN33
    JMP AGN3
;

AGN4: ; 输出一个峰值为5V的三角波
    MOV BX, 0
AGN41:
    MOV DX, 290H
    MOV AL, BL
    OUT DX, AL
    CALL DELAY

    CALL TRAN
    INC BL
    CMP BL, 255
    JB AGN41

AGN42:
    MOV DX, 290H
    MOV AL, BL
  
```

```

        OUT DX, AL
        CALL DELAY

        DEC BL
        CMP BL, 0
        JNE AGN42
        JMP AGN4

EXIT:
        MOV AH, 4CH
        INT 21H

;USER CODE END

DELAY PROC
        PUSH BX
        PUSH CX
        MOV BX, 10
NEXT5:  MOV CX, 10
NEXT:   LOOP NEXT
        DEC BX
        JNZ NEXT5

        POP CX
        POP BX
        RET
DELAY   ENDP

KEYIN PROC    ; 从键盘输入数字并显示，若不为
              数字则重新输入
BEG:
        MOV AH, 08H ;8号功能键入
        INT 21H
        CMP AL, 31H
        JB BEG
        CMP AL, 37H
        JA BEG
        MOV AH, 02H
        PUSH AX
        MOV DL, AL
        INT 21H
        POP AX
        RET
KEYIN ENDP

TRAN PROC    ; 跳转函数

        MOV DL, 0FFH
        MOV AH, 06H
        INT 21H

        CMP AL, 'q'
        JNE JP1
        JMP EXIT
JP1:    CMP AL, 'a'
        JNE JP2
        JMP AGN1
JP2:    CMP AL, 'b'
        JNE JP3
        JMP AGN2
JP3:    CMP AL, 'c'
        JNE JP4
        JMP AGN3
JP4:    CMP AL, 'd'
        JNE JP5
        JMP AGN4
JP5:
        RET
TRAN ENDP

CODE ENDS
END START

```

实验二 实验箱 ADC0809 读取电压值

;ADC0809输出端口地址

PADC0 EQU 2A0H

DATA SEGMENT

DT2 DB 4 DUP(0)

DATA ENDS

STK SEGMENT STACK

DW 100 DUP(?)

STK ENDS

CODE SEGMENT

ASSUME CS:CODE, DS:DATA, SS:STK

START:;初始化及数据段、堆栈段观察记录

MOV AX, STK

MOV SS, AX

MOV SP, 2*100

MOV AX, DATA

MOV DS, AX

AGN:

MOV DX, PADC0;启动AD转换

OUT DX, AL

CALL DELAY

IN AL, DX ;存入AL

MOV BL, AL

;16进制输出

PUSH AX

SHR AL, 4;高四位

AND AL, 0FH

CMP AL, 9

JBE NUM

ADD AL, 37H;数字A-F

JMP NEXT11

NUM:ADD AL, 30H;数字0-9

NEXT11:

MOV DL, AL

MOV AH, 02H

INT 21H

POP AX

PUSH AX

AND AL, 0FH;低四位

CMP AL, 9

JBE NUM1

ADD AL, 37H;数字A-F

JMP NEXT10

NUM1:

ADD AL, 30H;数字0-9

NEXT10:

MOV DL, AL

MOV AH, 02H

INT 21H

MOV DL, 'H'

MOV AH, 02H

INT 21H;输出H

MOV DL, 20H;输出空格

INT 21H

POP AX

MOV AL, 5

MUL BL ;乘以5后存入AX

MOV DX, 0

MOV BL, 0FFH

DIV BL ;AX/BL , 余数在AH, 商在AL

MOV DT2, AL; 个位数存入DT2[0]

MOV CX, 2

MOV DI, 1

LOOP2: MOV BL, AH; 小数点后2位分别存入DT2[1]和DT2[2]中

MOV AL, 10

MUL BL

MOV DX, 0

MOV BL, 0FFH

DIV BL

MOV DT2[DI], AL

INC DI

LOOP LOOP2

MOV CX, 3

MOV BX, 0

LOOP1:

MOV AL, DT2[BX] ;输出存入DT2中的

电压值

INC BL

ADD AL, 30H;转为ASCII码

MOV AH, 02H

MOV DL, AL

INT 21H

CMP BL, 1

JNE NEXT6

MOV DL, '.';输出小数点

MOV AH, 02H

INT 21H

```

NEXT6:      LOOP LOOP1
            MOV DL, 'V'
            MOV AH, 02H
            INT 21H;输出V

            MOV DL, 0DH;输出回车换行
            INT 21H
            MOV DL, 0AH
            INT 21H

            MOV AH, 08H;输出n退出
            INT 21H
            CMP AL, 'n'
            JE EXIT
            JMP AGN
    
```

```

EXIT:
            MOV AH, 4CH
            INT 21H
    
```

```

DELAY PROC
            PUSH BX
            PUSH CX

            MOV BX, 100
NEXT1:      MOV CX, 10
NEXT:
            LOOP NEXT
            DEC BX
            JNZ NEXT1

            POP CX
            POP BX
            RET
    
```

DELAY ENDP

```

CODE ENDS
END START
    
```

实验三 Proteus 仿真 DAC0832 数模转换

;想输出对应波形只需要注释掉其他波形的代码即可

```

DATA SEGMENT
    TBL DB
    128, 168, 203, 232, 250, 255, 250, 232, 203, 168, 128, 8
    8, 53, 24, 6, 0, 6, 24, 53, 88
    COUNT1 EQU $-TBL
    TBL2 DB 0, 255 ; 方波
    COUNT2 EQU $-TBL2
    
```

DATA ENDS

```

STK SEGMENT
    SSDAT DW 100 DUP(?)
STK ENDS
    
```

```

CODE SEGMENT
    ASSUME CS:CODE, DS:DATA, SS:STK
    
```

```

START:
    ;初始化及数据段、堆栈段观察记录
    MOV AX, STK
    MOV SS, AX
    MOV SP, 2*100
    
```

```

    MOV AX, DATA
    MOV DS, AX
    
```

```

    MOV BX, 0
    
```

AGN1: ;输出一个幅值为5V的正弦波

```

    MOV BL, 0
    
```

AGN11:

```

    MOV DX, 200H
    MOV AL, TBL[BX]
    OUT DX, AL
    CALL DELAY
    
```

```

    INC BL
    CMP BL, COUNT1
    JNE AGN11
    JMP AGN1
    
```

AGN2: ;输出一个方波

```

    MOV BX, 0
    
```

AGN22:

```

    MOV DX, 200H
    MOV AL, TBL2[BX]
    OUT DX, AL
    CALL DELAY
    
```

```

    INC BL
    CMP BL, COUNT2
    JNE AGN22
    JMP AGN2
    
```

AGN3: ;输出一个峰值为5V的锯齿波形

MOV BX, 0

AGN33:

MOV DX, 200H

MOV AL, BL

OUT DX, AL

CALL DELAY

INC BL

CMP AL, 255

JB AGN33

JMP AGN3

;

AGN4: ;输出一个峰值为5V的三角波

MOV BX, 0

AGN41:

MOV DX, 200H

MOV AL, BL

OUT DX, AL

CALL DELAY

INC BL

CMP BL, 255

JB AGN41

AGN42:

MOV DX, 200H

MOV AL, BL

OUT DX, AL

CALL DELAY

DEC BL

CMP BL, 0

JNE AGN42

JMP AGN4

EXIT:

MOV AH, 4CH

INT 21H

DELAY PROC

PUSH BX

PUSH CX

MOV BX, 10

NEXT5: MOV CX, 10

NEXT: LOOP NEXT

DEC BX

JNZ NEXT5

POP CX

POP BX

RET

DELAY ENDP

CODE ENDS

END START

实验四 Proteus 仿真 ADC0808 模数转换

;8255A地址

PORTA EQU 238H

PORTB EQU 23AH

PORTC EQU 23CH

PORTCTL EQU 23EH

;ADC0808地址

PADCO EQU 200H

DATA SEGMENT

DT1 DB ?

DT2 DB 4 DUP(0)

LIST DB 3FH, 06H, 5BH, 4FH, 66H, 6DH, 7DH, 07H,

7FH, 6FH

;七段码

DATA ENDS

STK SEGMENT STACK

SSDAT DW 100 DUP(?)

STK ENDS

CODE SEGMENT

ASSUME CS:CODE, DS:DATA, SS:STK

START:

MOV AX, STK ;初始化

MOV SS, AX

MOV SP, 2*100

MOV AX, DATA

MOV DS, AX

AGN:

MOV DX, PADCO ;启动模数转换

OUT DX, AL

CALL DELAY

IN AL, DX ;存入AL

MOV BL, AL

MOV AL, 5

MUL BL ;乘以5后存入AX

MOV DX, 0

MOV BL, 0FFH; BL=255

DIV BL ;AX/BL, 余数在AH, 商在

AL

MOV DT2, AL; 个位数存入DT2【0】

MOV CX, 2

MOV DI, 1

LOOP2:

MOV BL, AH; 小数点后2位分别
存入DT2[1]和DT2[2]中

MOV AL, 10

MUL BL; 商AH乘以10后存入AX

MOV DX, 0

MOV BL, 0FFH

```

        DIV BL ;AX/BL , 余数在AH, 商在AL

        MOV     DT2[DI],     AL
    INC     DI
    LOOP     LOOP2

        MOV     DX, PORTCTL
        MOV     AL, 88H      ;A, B, C端口都为输出
        OUT     DX, AL
        MOV     DX, PORTA    ;输出个位数
        MOV     BX, 0
        MOV     BL, DT2

        MOV     CH, LIST[BX]
        MOV     CL, 80H
        ADD     CL, CH;对七段码的第一位置1显示小数点

        MOV     AL, CL
        OUT     DX, AL
        MOV     DX, PORTC
        MOV     AL, 1011B
        OUT     DX, AL
        CALL    DELAY
        MOV     AL, 1111B
        OUT     DX, AL

        MOV     DX, PORTA;      输出小数点后一位
        MOV     BL, DT2[1]
        MOV     AL, LIST[BX]
        OUT     DX, AL
        MOV     DX, PORTC
        MOV     AL, 1101B
        OUT     DX, AL
        CALL    DELAY
        MOV     AL, 1111B
        OUT     DX, AL

        MOV     DX, PORTA    ;      输出小数点后
两位
        MOV     BL, DT2[2]
        MOV     AL, LIST[BX]
        OUT     DX, AL
        MOV     DX, PORTC
        MOV     AL, 1110B
        OUT     DX, AL
        CALL    DELAY
        MOV     AL, 1111B
        OUT     DX, AL

        MOV     AL, DT2[0]    ;比较电压值个位数
        CMP     AL, 1         ;小于1或者大于等于4, PB1输出高电平, 接通蜂鸣器。

        JB      ABB
        CMP     AL, 4
        JNB     ABB
        JMP     AGN

        ABB:
        MOV     DX, PORTB    ;PB1输出高电平, 蜂鸣器
        发出鸣响
        MOV     AL, 10B
        OUT     DX, AL
        CALL    DELAY1
        MOV     AL, 0B
        OUT     DX, AL
        JMP     AGN

    QUIT:    MOV     AH, 4CH
            INT     21H

    DELAY    PROC
            PUSH    BX
            PUSH    CX

            MOV     BX, 100
    NEXT1:    MOV     CX, 10
    NEXT:     LOOP   NEXT
            DEC     BX
            JNZ     NEXT1

            POP     CX
            POP     BX
            RET

    DELAY    ENDP

    DELAY1    PROC
            PUSH    BX
            PUSH    CX

            MOV     BX, 100
    NEXT5:    MOV     CX, 100
    NEXT4:    LOOP   NEXT4
            DEC     BX
            JNZ     NEXT5

            POP     CX
            POP     BX
            RET

    DELAY1    ENDP

    CODE     ENDS
    END      START
    
```